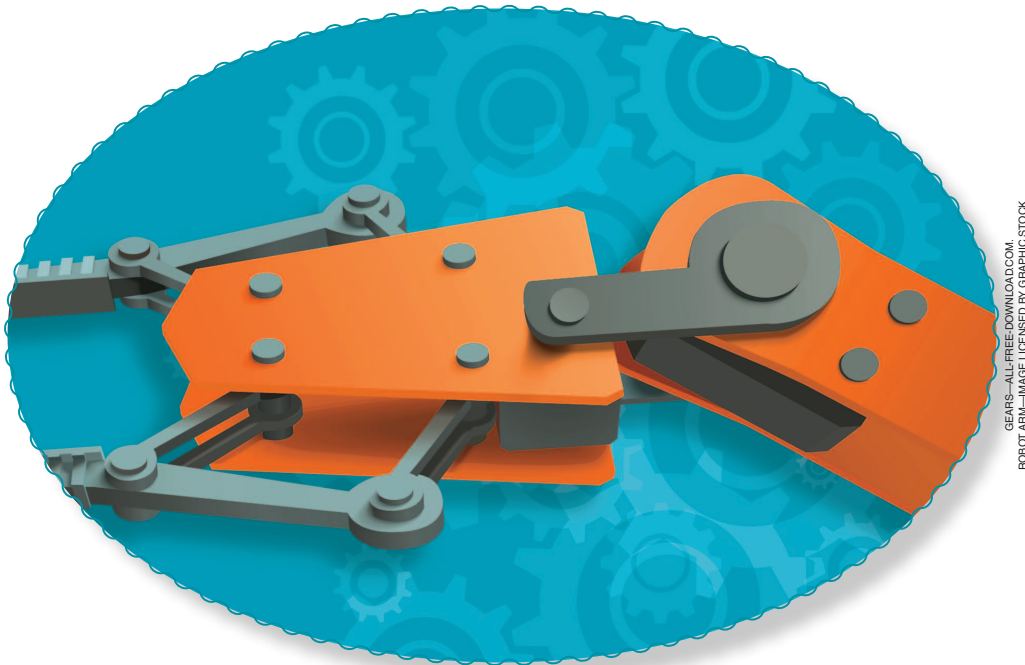


Controlling Kuka Industrial Robots

Flexible Communication Interface JOpenShowVar



GEARS—ALL-FREE-DOWNLOAD.COM.
ROBOT ARM—IMAGE LICENSED BY GRAPHIC STOCK

By Filippo Sanfilippo, Lars Ivar Hatledal,
Houxiang Zhang, Massimiliano Fago,
and Kristin Ytterstad Pettersen

JOpenShowVar is a Java open-source cross-platform communication interface to Kuka industrial robots. This novel interface allows for read-write use of the controlled manipulator variables and data structures. JOpenShowVar, which is compatible with all the Kuka industrial robots that use KUKA Robot Controller version 4 (KR C4) and KUKA Robot Controller version 2 (KR C2), runs as a client on a remote computer connected with the Kuka controller via TCP/IP. Even though only soft real-time applications can be implemented, JOpenShowVar opens up to a variety of possible applications, making the use of various input devices and sensors as well as the development of alternative control methods possible.

Four case studies are presented to demonstrate the potential of JOpenShowVar. The first two case studies are

open-loop applications, while the last two case studies describe the possibility of implementing closed-loop applications. In the first case study, the proposed interface is used to make it possible for an Android mobile device to control a Kuka KR 6 R900 SIXX (KR AGILUS) manipulator. In the second case study, the same Kuka robot is used to perform a two-dimensional (2-D) line-following task that can be used for applications, such as advanced welding operations. In the third case study, a closed-loop application is developed to control the same manipulator with a *Leap Motion* controller that supports hand and finger motions as input without requiring contact or touching. In the fourth case study, a bidirectional closed-loop coupling is established between a *Force Dimension* omega.7 haptic device and the same Kuka manipulator. Related experiments are carried out to validate the efficiency and flexibility of the proposed communication interface.

Digital Object Identifier 10.1109/MRA.2015.2482839
Date of publication: 10 December 2015

Communication Interfaces for Industrial Robots

Industries that employ robots in a wide variety of applications are the main customers for robot manufacturers. The manipulator market for research applications, on the other hand, is simply too small for the robot manufacturing industry to develop models specifically for such use. While the hardware and mechanical requirements of the developed robots are often similar for both industry and research, scientific software requirements are quite different and even contradictory in many aspects [1], [2]. The goal of scientists is to try to gain as much control over the robot as possible, whereas industries seek safe and easy operational interfaces.

In particular, although software interfaces that are appropriate for industrial use are available, it is difficult to find interfaces that are applicable for research purposes. The disclosure of the internal control architecture is also very hard to come by. Many manufacturers are unwilling to publish internal details regarding system architecture due to the high levels of competition in the robot market. Consequently, it is not possible to fully exploit many robotic platforms in a scientific context.

Only a small number of industrial manipulators with an open control interface have been released as far as robotics is concerned. Focusing exclusively on Kuka industrial robots [3], the Kuka robot language (KRL) is the standard programming language [4]. It is a text-based language that offers data type declaration, specification of simple motions, and interaction with tools and sensors via Input/Output (I/O) operations. It is only possible to run KRL programs on the Kuka robot controller (KRC), where program execution is done in accordance with real-time constraints. While the KRL offers an interface that is easy to use in industrial applications, it only has limited use for research purposes. In particular, the KRL is tailored to the underlying controller and consequently, only a fixed, controller specific set of instructions is offered [5]. Advanced mathematical tools, such as matrix operations, optimization, or filtering methods are not supported, and, thus, making the implementation of novel control approaches very difficult. There is no native way to include the third party libraries and as such, extending the KRL to include new instructions and functionalities is an arduous task. Moreover, it is not possible to directly use external input devices. The standard workaround for partially expanding the robot's capabilities is to use supplementary software packages provided by Kuka. Some examples of such packages are the Kuka.RobotSensorInterface [6], which allows the manipulator motion or program execution to be influenced by sensor data and the Kuka.Ethernet KRL XML [6], a module that allows the connection of the robot controller with up to nine external systems (e.g., sensors). However, several drawbacks accompany these supplementary software packages: I/O is limited, a narrow set of functions is present, and major capital investments are often required to purchase these packages from Kuka.

To overcome these problems, JOpenShowVar was presented by our research group in [7]. JOpenShowVar is a

Java open-source cross-platform communication interface that allows the user to read and write all of the controlled manipulator variables. Even though only soft real-time applications can be implemented, this interface allows researchers to use different input devices and sensors and to develop alternative control methods. The JOpenShowVar library is compatible with all the Kuka industrial robots that use KR C4 or KR C2. The basic concept is shown in Figure 1 where JOpenShowVar works as a middleware between the user program and the KRL. In this article, more details about the JOpenShowVar architecture are provided. Several new and more flexible and efficient procedures are introduced in the latest release of the library to replace the old fundamental reading and writing method that is now marked as deprecated. In addition to these new methods, some other high-level functions are also provided to enable angle and torque readings of the controlled manipulator. This feedback signal is an important factor in improving the manipulator dexterity and achieving crucial functions, such as sensitive collision detection and compliant control actions. Some guidelines for allowing the user to implement new high-level procedures are discussed. JOpenShowVar is an open-source project and it is available on the Internet at <https://github.com/aauc-mechlab/jopenshowvar>, along with several detailed class diagrams, documentation, and demo videos.

Related Research Work

The possibility of creating a software interface to Kuka robots has been investigated by several research groups. An open-source real-time control software for the Kuka lightweight robot, OpenKC, was presented in [1]. This software makes it possible to externally trigger and control all of the features of the Kuka lightweight (LBR) manipulator. This is done using a simple set of routines that can easily be integrated into existing software. As a result, developers of robot applications have an edge in finding solutions for a variety of different software scenarios. In particular, force and torque readings as well as different modes of operation can be remotely read and parameterized. However, this software interface is restricted to a specific model of Kuka robots, the Kuka lightweight manipulator and use of the Kuka.RobotSensorInterface package is required. Another interface that is currently available for the Kuka lightweight robots is the fast research interface (FRI) [2]. The FRI provides direct low-level real-time access to the KRC at rates of up to 1 kHz. On the other hand, all features, such as teaching, motion script features, fieldbus I/O, and safety are provided. The FRI is based on the KR C2.

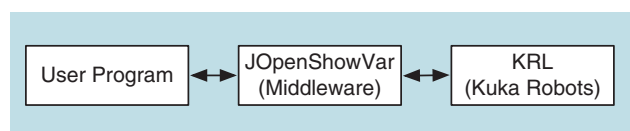


Figure 1. The idea of realizing a communication interface for Kuka industrial robots that works as a middleware between the user program and the KRL.

Without burdensome installation efforts, access to different controller interfaces of the Kuka system is provided, including joint position control, cartesian impedance control, and joint position control. However, this software interface is restricted to a specific model of Kuka robots, the Kuka lightweight manipulator. No support for the standard Kuka industrial robots is provided.

The Kuka control toolbox (KCT), a collection of MATLAB functions for motion control of Kuka industrial robots, was introduced in [10] to offer an intuitive and high-level programming interface for the user. This toolbox is compatible with all small and low-payload Kuka robots that have six degrees of freedom (6 DOF). The KCT runs on a remote computer connected to the KRC via

TCP/IP. A multithread server runs on the KRC and communicates via Kuka.Ethernet KRL XML with a client whose job is to manage the information exchange with the manipulator. High-transmission rates are guaranteed by this communication setup, and thus enabling real-time control applications. Nonetheless, as in the previous work, this approach

is still tailored to the underlying controller and requires the use of the Kuka.Ethernet KRL XML package.

A different approach has been tried by the other researchers, aimed at the disclosure of the Kuka industrial manipulator internal control architecture. For instance, the reverse engineering of the KRL was investigated in [5] and a set of Java-based robotics APIs was presented for programming industrial robots on top of a general-purpose language. The robotics APIs implement robot commands like motions and access to

I/O calls. It was shown that KRL can be bridged by batch-executing motions, under the assumption that executing control flow and calculation statements takes only a small amount of time compared with the time it takes the robot to complete a motion command. However, some safety limitations are inherently present in the robotics APIs set because it is the result of a reverse engineering approach and therefore does not include a way of specifying complex triggers in contrast to the KRL.

In the last few years, the robot operating system (ROS) [11], an open-source software toolbox for robotic development, has become more and more popular among the research community. The primary goal of ROS is to provide a common platform to make the construction of capable robotic applications quicker and easier. Some of the features it provides include hardware abstraction, device drivers, message-passing, and package management. The provides support for different industrial robots, including vendors like ABB, Adept, Fanuc, Motoman, and Universal Robots. Extensive research work has also gone into creating ROS packages to communicate with the Kuka lightweight robots but no support is provided for the Kuka standard industrial robots yet. One of the main reasons for this is the nondisclosure of the KRC internal architecture, which currently makes it impossible to directly interact with the robot to be controlled.

Recently, Kuka has shown more interest in the research and education market. In particular, the KUKA Sunrise.Connectivity has been recently developed for Kuka lightweight robots. This software provides a collection of interfaces for influencing robot motion at various process control levels. The third-party software can be easily integrated into the user-specific application using the popular standard programming language Java. Along with the quick update of the target position directly from the robot application, it is also possible to access the robot controller from external computers in hard real-time mode. However, even in this last case, the main limitation is that this software is restricted to the Kuka lightweight manipulators.

To provide a clear overview of the currently available interfaces for Kuka robots, a table of comparison of all the reviewed related works is shown in Table 1. To the best of our knowledge, a cross-platform communication interface that works with all Kuka industrial robots without requiring any external packages has not been released.

JOpenShowVar Architecture

The authors initially describe the design choices that characterize the proposed architecture. Successively, the architectural concept is presented, the communication protocol is analyzed, possible control approaches and some high-level methods are explored.

The design of JOpenShowVar is based on the following design choices.

- **Low Cost:** The developed architecture does not require supplementary software packages provided by Kuka, such as the Kuka.RobotSensorInterface [6] or the Kuka.Ethernet KRL XML [6]. Therefore, no major capital investments are

Even though there is a delay between the input signal and the actual position, the results show that the system is quite responsive to the user's inputs.

Table 1. The interfaces currently available for Kuka robots.

Interface	Support to Kuka LBR	Support to Kuka Industrial Robots	External Packages Required
OpenKC	Yes	No	Yes
FRI	Yes	No	No
KCT	No	Yes (only small and low payload)	Yes
Robotics APIs	Yes	Yes (safety limitations)	No
ROS	Yes	No	No
KUKASunrise. Connectivity	Yes	No	Yes

required to actually purchase these packages from Kuka, which makes the proposed solution very inexpensive.

- **Flexibility:** The system offers a virtually unlimited I/O and the possibility of including third party libraries. This allows the user to add support for advanced mathematical tools such as matrix operations, optimization, or filtering methods, making it very simple to implement novel control approaches.
- **Reliability:** The system is easy to maintain, modify and expand by adding new components and features. In addition the proposed interface is also open-source and cross-platform.
- **Integrability:** The proposed system interface presents a modular structure that can facilitate the integration with ROS. Even though this integration is outside the scope of this article, it is considered an important future work which will surely improve the usefulness of the proposed interface. The community of developers at ROS is looking forward to the integration of JOpenShowVar. The developers have confirmed the usefulness of the proposed interface because there are currently no other alternatives offering similar features.

Hereafter, several specific functions, variables, and configurations related to the KRL and the KRC are used to introduce the architectural concept. For a more detailed introduction to the KRL, the reader can refer to [4]. Figure 2 shows the proposed control system architecture. It is a client-server architecture with JOpenShowVar running as a client on a remote computer and KUKAVARPROXY acting as a server on the KRC. The JOpenShowVar locally interacts with the user program and remotely communicates with the KUKAVARPROXY server via TCP/IP.

In particular, KUKAVARPROXY is a multient server that is written in Visual Basic 6.0 and can serve up to ten clients simultaneously. KUKAVARPROXY implements the Kuka CrossComm interface. This interface allows for the interaction with the real-time control process of the robot and makes it possible to perform several operations, such as selection or cancellation of a specific program, errors and faults detection, renaming program files, saving programs, resetting I/O drivers, reading variables, and writing variables. KUKAVARPROXY implements the reading and writing methods. All variables that need to be accessed by these methods have to be declared as global variables in the predefined global system data list \$CONFIG.DAT. Various variables can be declared in this file from basic types, such as INT, BOOL, and REAL, to more complex structures like E6POS and E6AXIS that allow storage of the robot configuration. Moreover, several system variables can be accessed and provided that there are no restrictions due to the type of data, such as for \$PRO_IP, \$POS_ACT, \$AXIS_ACT, or \$AXIS_INC. For example, the current robot position, \$POS_

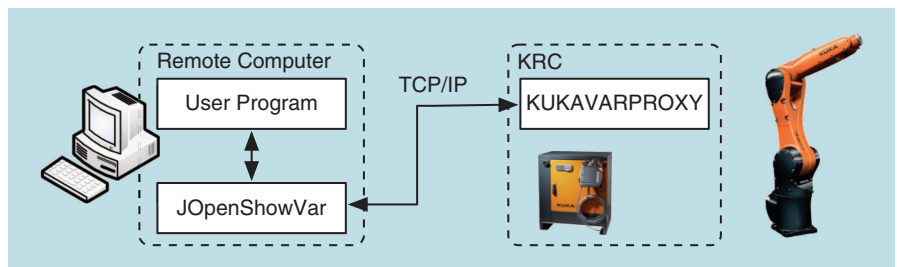


Figure 2. The proposed architecture for JOpenShowVar where a client-server model is adopted. (Photo courtesy of KUKA Robotics Corporation.)

ACT, cannot be written but only read. Restrictions of this nature are checked by the controller.

As already mentioned, the interface of the Kuka CrossComm class allows for the interaction with the real-time control process of the robot to be controlled. However, it should be noted that the Kuka CrossComm class can only be remotely accessed via TCP/IP. Unfortunately, the TCP/IP communication introduces inevitable delays, therefore, JOpenShowVar cannot provide a real-time access to the robot's data. Only soft real-time applications can be realized. In fact, it takes a nondeterministic time to access a specific variable. Since Kuka does not offer any kind of documentation on this topic, several experimental tests were performed at our laboratory to assess this time interval. According to our experiments, reported in the "Experimental Results" section, the average access time is about 5 ms. Moreover, this time interval is not affected by the kind of access to be performed (whether it is a reading or a writing operation) or by the length of the message. For these reasons, it is advantageous to aggregate several variables in logical structures when reading or writing data. By using data structures, it is possible to simultaneously access several variables and minimize the access time. The only limitation to this approach is on the length of the logical structures that cannot exceed 255 B.

JOpenShowVar provides a client, CrossComClient, which is written in Java, thus making cross-platform support possible. The architectural details of the JOpenShowVar library are shown in Figure 3. As presented in [7], the client initially provided only one low-level method, sendRequest. This method allows for both reading and writing variables. The sendRequest method returns a callback instance containing the updated value. However, in the latest release of JOpenShowVar, starting from version v0.2, the sendRequest is marked as a deprecated method, since two new more flexible and efficient methods are introduced: readVariable and writeVariable.

On top of the JOpenShowVar's methods that implement the low-level communication protocol, another logic layer can be

The aim is to show the possibility of operating the robot and transferring the corresponding force feedback to the operator.

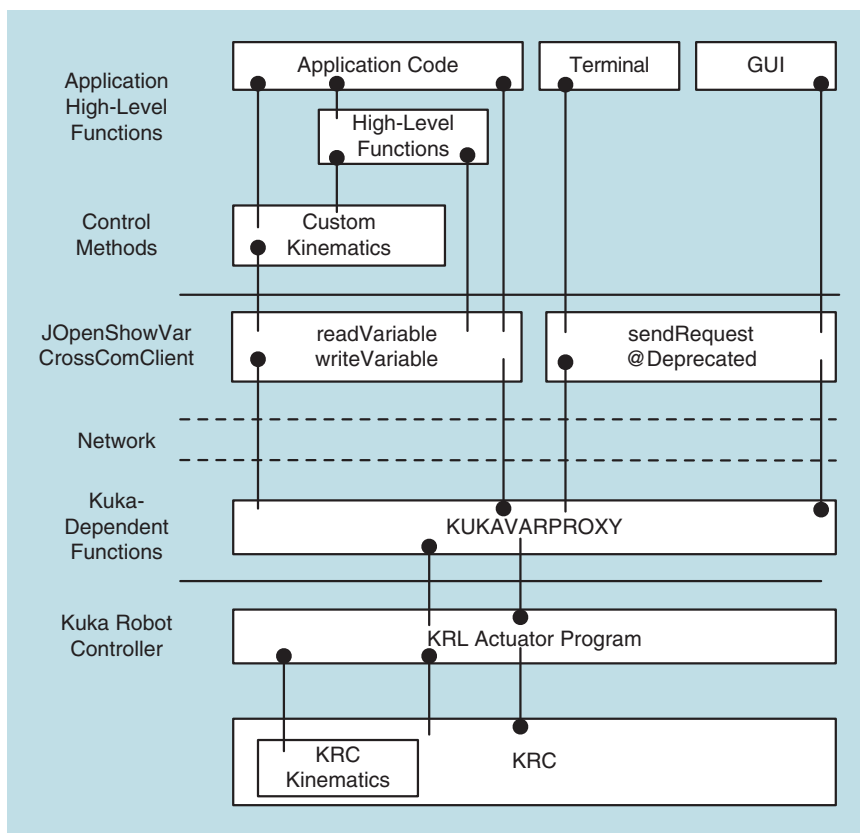


Figure 3. The architectural levels of JOpenShowVar.

added by the user developer allowing for the possibility of implementing alternative control methods (custom kinematics) as well as some higher-level functions. The application code can run on top of this hierarchical architecture. In addition, a graphical user interface (GUI) and a terminal are provided with JOpenShowVar to allow the user to monitor the robot's state, visualize, and manually set all the desired variables. It should be noted that the GUI still uses the `sendRequest` method of JOpenShowVar for a practical reason, since this old method does not require knowledge on the internal structure of the variables to be accessed compared with the new methods. The low-level communication protocol, a detailed reference explanation of the newly released methods, the possibility of implementing custom control functions, as well as some guidelines to develop high-level procedures will be discussed later in this section.

Table 2. The reading variables.

Field	Description
00	Message ID
09	Length of the next segment
0	Type of desired function
07	Length of the next segment
\$OV_PRO	Variable to be read

Communication Protocol

The communication protocol relies on the TCP/IP protocol. In particular, on top of the TCP/IP layer, specially formatted text strings are used for message exchanges. KUKAVARPROXY actively listens on the TCP port 7000. Once the connection is established, the server is ready to receive any reading or writing request from the client.

Reading Variables

To access a variable, the client must specify two parameters in the message: 1) the desired type of function and 2) the variable name. To read a specific variable, the type of function must be identified by the character 0. For instance, if the variable to be read is the system variable `$OV_PRO`, which is used to override the speed of the robot, the message that the client has to send to the server will have the format shown in Table 2. In detail, the first two characters of this string specify the message identifier (ID) with a progressive integer number between 00 and 99. The answer

from the server will contain the same ID so that it is always possible to associate the corresponding answer to each request even if the feedback from the server is delayed. The next two characters in the string specify the length of the next segment in hexadecimal units. In this specific case, nine accounts for one character specifying the function type, two characters indicating the length of the next segment, and seven characters for the variable length. The fifth character 0 in the message represents the type of the desired function, which in this case is reading. Subsequently, there are two more characters indicating the variable length (in hexadecimal units) and finally the variable itself is contained in the last section of the message.

Writing Variables

To write a specific variable, three parameters must be specified: 1) the type of function, 2) the name of the desired variable, and 3) the value to be assigned. The writing function is specified by the character 1. For instance, if the variable to be written is the system variable `$OV_PRO` with a value of 50 (50% override speed), the message that the client has to send to the server will have the format shown in Table 3.

Variables, Structures, and Methods

With the release version v0.2 of JOpenShowVar, several new classes have been added to the library to improve the usability. In particular, two abstract classes: the `KRLVariable` and the `KRLStruct` (which extends `KRLVariable`) are provided to

allow the user to implement any KRL variable or structure, respectively. Using this method, it is possible to create and maintain a local instance of all the desired variables and structures on the client side. Based on these two abstract classes, the most commonly used KRL variables and structures have been implemented. Any other KRL variable or structure that is not included in the JOpenShowVar library can be easily implemented by the user.

Since the release of version v0.2 of JOpenShowVar, the `sendRequest` is marked as a deprecated method. To replace this old method, two new and more reliable methods are added to the CrossComClient as follows.

- The `readVariable` method allows the user to read any desired remote variable or structure from the controlled robot and store it locally. An exception is thrown if an error in the communication protocol occurs.
- The `writeVariable` method allows the user to update any desired remote variable or structure of the controlled robot with the value of the corresponding local variable or structure, respectively. An exception is thrown if an error in the communication protocol occurs.

The deprecated `sendRequest` method is being kept as part of the JOpenShowVar library simply because the GUI still uses it. In fact, this old method does not require any knowledge on the internal structure of the variables to be accessed compared with the newly introduced methods. Moreover, it should be noted that the new method `writeVariable` cannot handle arrays; this can only be done using the old `sendRequest` method. In the Algorithm 1, a possible use-case example is shown to highlight the differences between the new methods and the deprecated `sendRequest` method.

Control Methods

JOpenShowVar opens up a variety of possible applications that make it possible to use different input devices and develop alternative control methods. In particular, the proposed interface provides the possibility of implementing either a position or a velocity control approach. The user experience is substantially different in each case. When using the position control mode, the operator simply controls the position of the robot's end-effector with constant velocity; when operating in velocity control mode, the operator also sets the velocity of

the robot tool. In the first case, when the operator releases the input device, the end-effector moves back to its starting point, while in the second scenario, the arm just stops moving but it keeps the last given position.

To control the robot motion according to the desired operational scenario, JOpenShowVar allows researchers to use the standard kinematics provided with the KRC. However, it is also possible to implement alternative control algorithms according to current needs. This is shown in Figure 4(a) and (b), respectively. It should be noted that the KRL does not provide a native way to obtain velocity control. When using the KRC kinematics, this limitation can be overcome by expressing the target position as:

$$\mathbf{x}_t = \mathbf{x}_d, \quad (1)$$

if operating in position control mode, or by:

$$\mathbf{x}_t = \mathbf{x}_d + \dot{\mathbf{x}}_d \Delta t, \quad (2)$$

if operating in velocity control mode, where Δt is the estimated time interval between two successive iterations. As previously mentioned, JOpenShowVar cannot provide a real-time access to the robot's data. Only soft real-time applications can be realized. It takes a nondeterministic time to access a specific variable. According to our experiments, reported in the "Experimental Results" section, the average access time is about 5 ms. Therefore, Δt can be approximated to a slightly bigger factor of the average access time. To achieve better performance, the average access time should be continuously monitored and updated. Perhaps, this may be a price too high to pay for some applications with real-time requirements, but JOpenShowVar still provides great advantages in terms of flexibility.

Alternatively, when a custom control algorithm is needed, the target joint configuration is given by:

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_d, \quad (3)$$

Algorithm 1. A use-case example that highlights the differences between the new methods and the deprecated `sendRequest` method.

```
try (CrossComClient client = new CrossComClient
    ("robotIPAddress", 7000)) {
    //JOpenShowVar v0.1 reading
    Callback readRequest = client.
        sendRequest(new Request(0, "$OV_JOG"));
    System.out.println(readRequest);
    //JOpenShowVar v0.1 writing
    Callback writeRequest = client.
        sendRequest(new Request(1, "$OV_JOG",
        "100"));
    System.out.println(writeRequest);
    //JOpenShowVar v0.2 reading
    KRLReal jog = KRLVariable.OV_JOG();
    client.readVariable(jog);
    System.out.println(jog);
    //JOpenShowVar v0.2 writing
    jog.setValue(10);
    client.writeVariable(jog);
    System.out.println(jog);}
```

Table 3. The writing variables.

Field	Description
00	Message ID
0b	Length of the next segment
1	Type of desired function
09	Length of the next segment
\$OV_PRO	Variable to be written
50	Value to be written

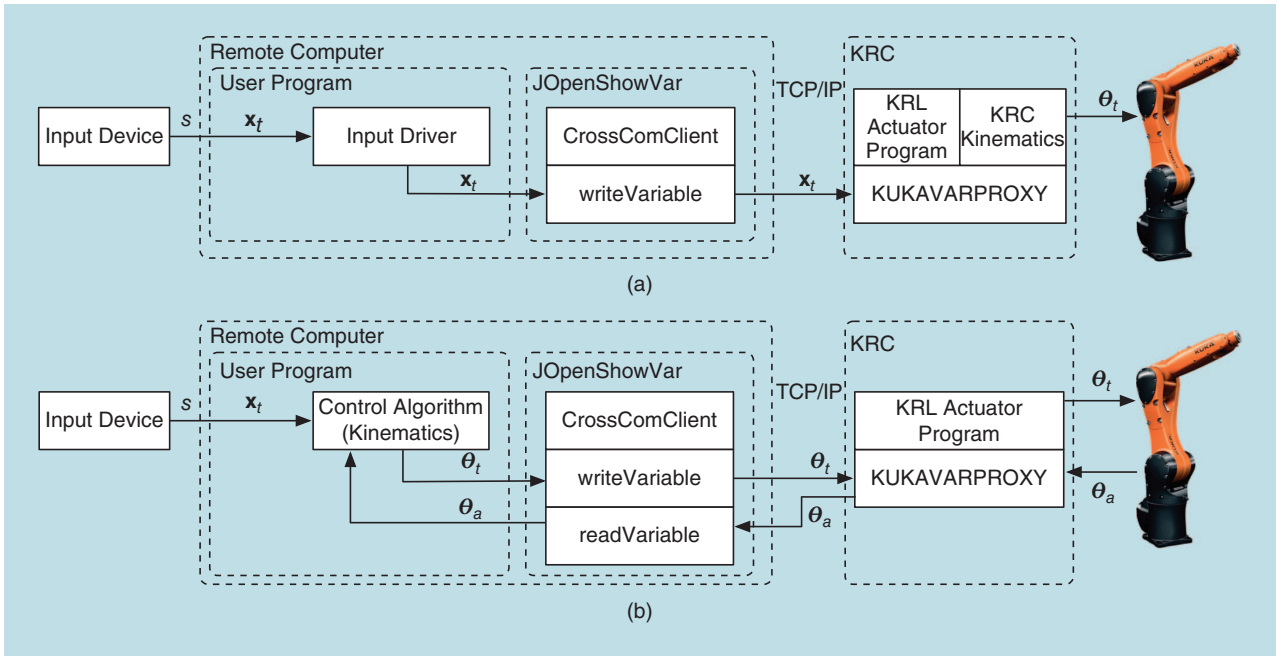


Figure 4. (a) The user program utilizes JOpenShowVar to set the desired end-effector position and then the robot joints are calculated by the KRC using the standard kinematic model. (b) A custom control algorithm can be implemented by the user to calculate the joint values for the robot and then send these angles to the KRC to be actuated. (Photo courtesy of KUKA Robotics Corporation.)

if operating in position control mode, or by:

$$\dot{\theta}_d = f_v^{-1}(\theta_a, \dot{x}_d), \quad (6)$$

$$\theta_t = \theta_a + \dot{\theta}_d \Delta t, \quad (4)$$

if operating in velocity control mode.

When the operator maneuvers the manipulator, a vector signal with no semantic, s , is sent from the input device to the user program. Here, according to the operational scenario, the vector signal is interpreted as the target position x_t . If the intent is to use the standard kinematics provided with the KRC, the user program simply works as a driver for the input device and uses the writeVariable method of JOpenShowVar to forward x_t to a KRL program where the standard KRC kinematics is used to calculate the joint angles θ_d . Alternatively, a custom control algorithm can be implemented within the user program to calculate the joint values for the robot according to x_t . Essentially, the custom control method has to implement classic inverse kinematic functions that can be generalized as follows:

$$\theta_d = f_p^{-1}(x_d), \quad (5)$$

concerning position control, and

for velocity control, where θ_a is the actual joint angles vector that can be retrieved using the readVariable method of JOpenShowVar. These values are then forwarded to a KRL program where the standard KRC functions are used to actuate the robot.

Note that the possibility of implementing certain control features does not influence the design for the presented interface. Instead, JOpenShowVar extends the functionalities of the KRL language.

Additional Functions

To simplify the low-level communication protocol and improve reliability, some additional methods are provided with the CrossComClient class as follows.

- The simpleRead and the simpleWrite methods are simpler versions of the sendRequest function. In particular, these methods do not execute any data parsing as opposed to the sendRequest method. They allow for an easier and faster access, as shown in Algorithm 2. The two new methods return a raw string without parsing the information. The aim of these two new methods is to provide an easy way to monitor the status of the robot making it possible to print the raw information returned from the KRC.
- The readJointAngles method uses the readVariable method to retrieve the actual joint angles vector, θ_a , of the controlled robot, all at once.
- The readJointTorques method allows for monitoring the load of each joint actuator by retrieving the current torque of each axis of the arm, all at once. In particular, readJointTorques retrieve the global KRL array variable

Algorithm 2. A use case for the new simpleRead and simpleWrite methods.

```
try (CrossComClient client = new CrossComClient
    ("robotIPAddress", 7000)) {
    System.out.println(client.simpleRead("$OV_
    JOG"));
    System.out.println(client.
        simpleWrite("$OV_JOG", "90"));}
```

Algorithm 3. Reading the actual torque for each axis, Java side.

```
try (CrossComClient client = new CrossComClient
    ("robotIPAddress", 7000)) {
    double[] torques = client.readJoint-
        Torques();}
```

\$TORQUE_AXIS_ACT and returns the current torque of each axis. This feedback signal is very important to improve the manipulator dexterity and to achieve the crucial functions, such as sensitive collision detection and compliant control actions. In Algorithm 3, a possible use-case is shown.

In addition to these methods, other high-level functions can be implemented by the user on top of the JOpenShowVar communication protocol. The implementation of other possible high-level applications is included as a technical document in the public repository of JOpenShowVar.

Terminal and GUI

Another useful tool that comes with JOpenShowVar is a console terminal that provides read-write text-based access to the robot's data. It is particularly useful for system administration and debugging purposes. To read a variable, it is sufficient to type the name of the desired variable and press enter. From an implementation point of view, it uses the new simpleRead and simpleWrite methods. Figure 5(a) shows a simple use-case.

In addition, JOpenShowVar also offers a useful GUI that can be used to monitor the robot's state, visualize, and manually set variables. A screenshot of this convenient tool is shown in Figure 5(b). This interface is very intuitive for the user.

Case Studies

Four case studies are presented to demonstrate the potential of JOpenShowVar. The first two case studies are open-loop applications, while the last two case studies describe the possibility of implementing closed-loop applications.

Case Study 1

To show the potential of the presented interface in controlling a Kuka industrial robot from an alternative input device, as a first case study, JOpenShowVar is used to control a Kuka KR 6 R900 SIXX manipulator with an Android mobile device. In this case, an open-loop application is implemented using the standard kinematics provided with the KRC. The Kuka KR 6 R900 SIXX, shown in Figure 6(a), is a 6 DOF robotic arm with a slim design and a small footprint.

According to the operational scenario, an Android mobile application whose GUI is shown in Figure 6(b), is used to set the target position $\mathbf{x}_t$. Using the writeVariable method of, JOpenShowVar, this vector is forwarded to the KUKAVARPROXY and stored as a global value in a data structure. Finally, a KRL actuator program iteratively retrieves the

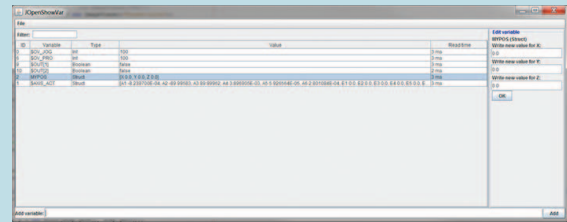
```
Aalesund University College - JOpenShowVar Console Application
(C) 2014 Aalesund University College
All Rights Reserved.
Author - Lars Ivar Hatledal [laht@hials.no]

Read variable: Type the name of the variable you want to read and hit enter.
Example: $0V_JOG
Write variable: Type the name of the variable followed by the new value to write
and hit enter.
Example: $0V_JOG 90
Example: MYPOS {X 0.2,Y -0.5,Z 0}
Exit by typing 'q'

Insert command:
$0V_JOG
Got: Callback{variable=$0V_JOG, id=0, option=0, readTime=2ms, value=10}

Insert command:
$0V_JOG 50
Got: Callback{variable=$0V_JOG, id=0, option=1, readTime=3ms, value=50}
```

(a)

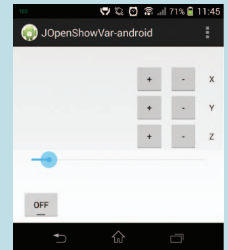


(b)

Figure 5. (a) The JOpenShowVar terminal can be used for debugging purposes. (b) The JOpenShowVar GUI can be used to monitoring the robot's state, visualize, and manually set variables and structures.



(a)



(b)

Figure 6. Case study 1: (a) The Kuka KR 6 R900 SIXX manipulator. (b) The GUI of the Android mobile application used to control the arm. (Photo courtesy of KUKA Robotics Corporation.)

Algorithm 4. The KRL actuator program for case study 1.

```
DEF ACTUATOR()
INI
PTP HOME Vel = 100 % DEFAULT
$ADVANCE = 1
LOOP
    PTP_REL MYPOS C_PTP
ENDLOOP
PTP HOME Vel = 100 % DEFAULT
END
```

new global data and uses the KRC kinematics to actuate the robot. The code of the KRL actuator program is shown in Algorithm 4. For Kuka industrial robots, the idle time between motions can be shortened by executing the time-consuming

arithmetic and logic instructions between motion commands while the robot is moving, i.e., processing them during the advance run (the instructions run in advance of the motion). Using the system variable \$ADVANCE, it is possible to define the maximum number of motion blocks that the advance run can process ahead of the main run (the motion block currently being executed). Since the main loop of the server program consists of only one instruction, the system variable \$ADVANCE is initially set to one to avoid the unwanted execution of the same line of code. Inside the main loop, a relative movement is iteratively executed to the global variable MYPOS, which is the one that stores the target position. The key word C_PTP is used to approximate the movement. The approximate positioning instruction is executed in a time-optimized manner: there is always at least one axis moving with the programmed acceleration or velocity limits. The system simultaneously ensures that the permissible gear and motor torques for each axis are not exceeded. Furthermore, the higher motion profile, set by default, ensures motion that is optimized in terms of velocity and acceleration.

Case Study 2

In this case study, JOpenShowVar is adopted to perform a 2-D line-following task with the same Kuka robot used in the previous example. In this case, an open-loop off-line application is implemented using the standard kinematics provided with the KRC. The considered task can be used for applications such as advanced welding operations. In this experiment, a camera is mounted on the robot's end-effector and can capture a photo of the desired line to be followed on a

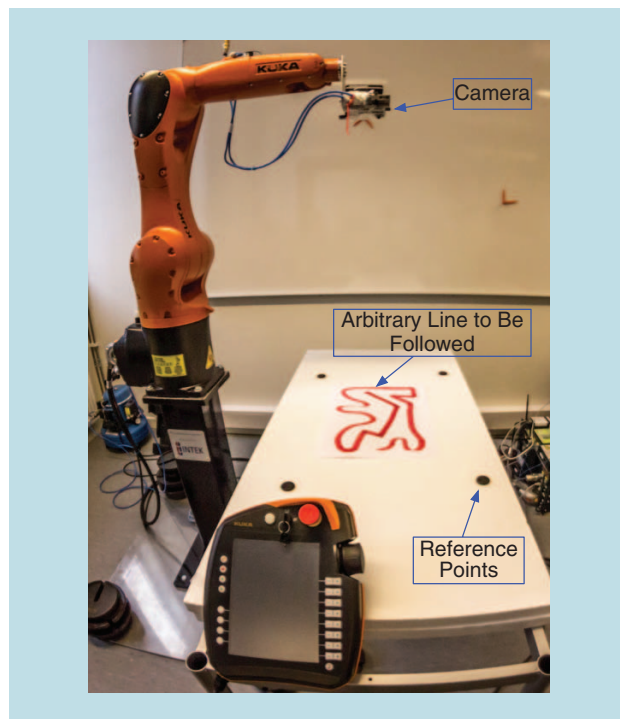


Figure 7. Case study 2: the experiment setup for a 2-D line-following task with the Kuka KR 6 R900 SIXX manipulator. (Photo courtesy of KUKA Robotics Corporation.)

plane. This vision feedback is used for the off-line detection of the path before starting the movement. In particular, once a photo of the line to be followed is taken, the operator manually selects the desired initial and final points. Then, the A^* search algorithm [12] is used to efficiently find a traversable path between these two points within the region covered by the desired line. The detected traversable path is sampled with a predefined resolution and the resulting samples are stored in an array variable. The same array is used as an offline input for the robot's end-effector to be actuated point by point. The experiment setup is shown in Figure 7.

Case Study 3

In this case study, JOpenShowVar is used to control the same robot (from the previous case studies) in a closed-loop and with a custom control algorithm. This is done to highlight the potential of the presented interface in developing alternative control methods that do not use the standard kinematic model provided by Kuka. Moreover, a *Leap Motion* controller [9], shown in Figure 8, is used as an alternative input device to control the robot. The *Leap Motion* controller is a small universal serial bus (USB) input device that supports hand and finger motions as input without requiring contact or touching. This controller is designed to be placed on a physical desktop, facing upward. Using two monochromatic infrared (IR) cameras and three IR light-emitting diodes (LEDs), the device observes a roughly hemispherical area, to a distance of about 1 m. The LEDs generate a three-dimensional pattern of dots of IR light and the cameras generate almost 300 frames/s of reflected data, which is then sent through a USB cable to the host computer. Next, it is analyzed by the *Leap Motion* controller software and can be retrieved using the *Leap Motion* APIs. While the *Leap Motion* controller makes it possible to control all of the joints in human hands, in this specific case study, only the DOF of the wrist are used as an input signal to control the robot's end-effector. Each DOF of the wrist corresponds to a translational axis in the workspace of the robot to be controlled. When operating in position control mode, the input device works as a position proportional replica, so that the wrist motion maps exactly to the motion of the robot's end-effector with constant speed, while, when

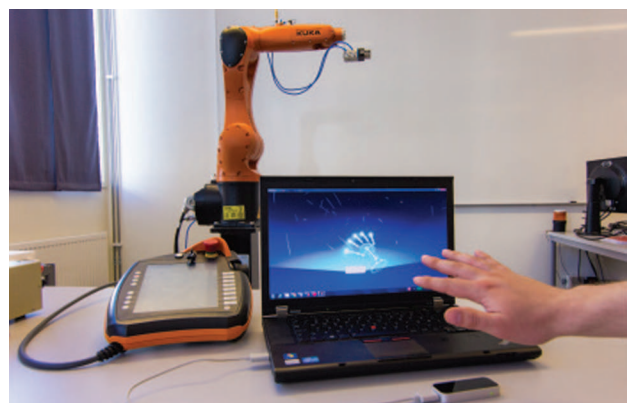


Figure 8. Case study 3: the *Leap Motion* controller used to operate the Kuka KR 6 R900 SIXX manipulator. (Photo courtesy of KUKA Robotics Corporation and Leap Motion Inc.)

operating in velocity control mode, a movement of the wrist in a particular direction will produce a translational motion in the same direction at a velocity proportional to the wrist displacement. For small vibrations not to affect the motion of the robot's end-effector, a small spherical imaginary volume with a diameter of about 8 cm is defined in the center of the controller monitoring space. As long as the operator's wrist is located within this volume, the robot's end-effector does not move. The operator's hand has to be moved more than 4 cm from the center of the monitoring space in order to generate a motion. Because of the modularity of the architecture, any other joystick or input device can be used without influencing the effectiveness of the proposed interface.

The user program runs on a remote computer and uses the *Leap Motion* APIs to retrieve the target position $\mathbf{x}_t$ according to the operational scenario. Using the `readVariable` method of `JOpenShowVar`, the actual joint angles θ_a are received. This data are used as input for the custom control algorithm. In this specific case study, the classical kinematic functions and the Jacobian method [13] are used to implement (5) and (6). Then, using the `writeVariable` method of `JOpenShowVar`, the target joint configuration θ_t is forwarded to the KUKAVARPROXY and stored as a global value in a structure. Finally, a KRL actuator program iteratively retrieves the new global data and actuates the robot.

The code of the KRL actuator program is shown in Algorithm 5. It should be noted that the variable MYAXIS is initialized to default values inside the initial fold. The system variable \$ADVANCE is initially set to 1. Then, the current joint values are assigned to a local structure variable named LOCAL. Inside the main loop, the desired joint angles are iteratively assigned to LOCAL, axis by axis. Finally, a PTP movement with C_PTP approximation is executed.

Case Study 4

The aim of this fourth case study is to show the possibility of operating the robot and transferring the corresponding force feedback to the operator. For this purpose, a bidirectional coupling between a *Force Dimension* omega.7 [19] haptic device and the same Kuka robot used in the previous sections are established. In this case, a closed-loop application is implemented using the standard kinematics provided with the KRC. The omega.7 is a 7 DOF haptic interface with high precision active grasping capabilities and orientation sensing. Finely tuned to display perfect gravity compensation, the force-feedback gripper offers extraordinary haptic capabilities, enabling instinctive interaction with complex haptic applications. In this case study, the principle of virtual works [13] is applied. According to this principle, the following equation is valid:

$$J^T \mathbf{F} = \boldsymbol{\tau}, \quad (7)$$

where J is the Jacobian matrix of the arm, $\mathbf{F}$ is the vector of forces exerted from the robot's end-effector to the environment and $\boldsymbol{\tau}$ is the vector of torques at the joints that can be retrieved using the `readJointTorques` method. By applying this

Algorithm 5. The KRL actuator program for case study 3.

```
DEF EXT_MOVE_AXIS()
  DECL AXIS LOCAL
  INI
  PTP HOME Vel = 100 % DEFAULT
  $ADVANCE = 1
  LOCAL.A1 = $AXIS_ACT.A1
  ...
  LOCAL.A6 = $AXIS_ACT.A6
  LOOP
    LOCAL.A1 = LOCAL.A1 + MYAXIS.A1
    ...
    LOCAL.A6 = LOCAL.A6 + MYAXIS.A6
    PTP LOCAL C_PTP
  ENLOOP
  PTP HOME Vel = 100 % DEFAULT
END
```

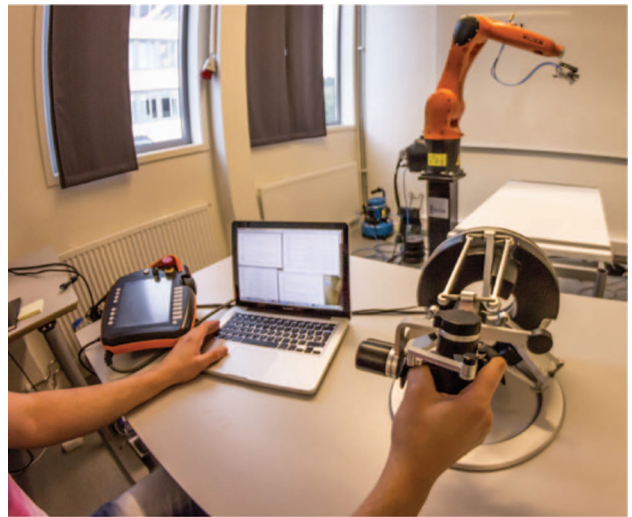


Figure 9. Case study 4: controlling the Kuka KR 6 R900 SIXX manipulator with an omega.7 haptic device from *Force Dimension*. (Photo courtesy of KUKA Robotics Corporation, Leap Motion Inc., and Force Dimension [number of the newly added reference].)

principle, it is possible to simulate on the haptic device a force feedback proportional to the force that the robot's end-effector is supporting. The experimental setup is shown in Figure 9.

Experimental Results

Experiments related to the proposed case studies are carried out to test the proposed communication interface in terms of accuracy, performances, and effectiveness. Concerning the first and the third case studies, a demo video is available online at: <https://youtu.be/fC8jb9MKgGw>. The video is also available as a multimedia supplement on *IEEE Xplore*.

The first case study highlights the potential of `JOpenShowVar` in remotely controlling a Kuka industrial robot from an Android mobile device. This possibility opens up to a variety of useful purposes including human interface applications and teleoperations. Nowadays, smartphones and tablets are becoming computationally more and more powerful. In this perspective, they are a perfect match with robots for

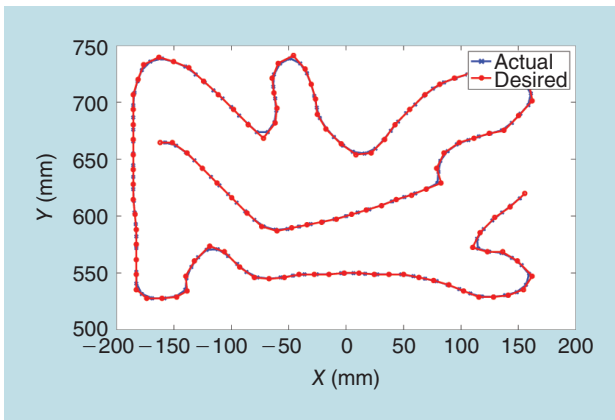


Figure 10. Case study 2: the detected line and the actual path followed by the robot's end-effector, respectively.

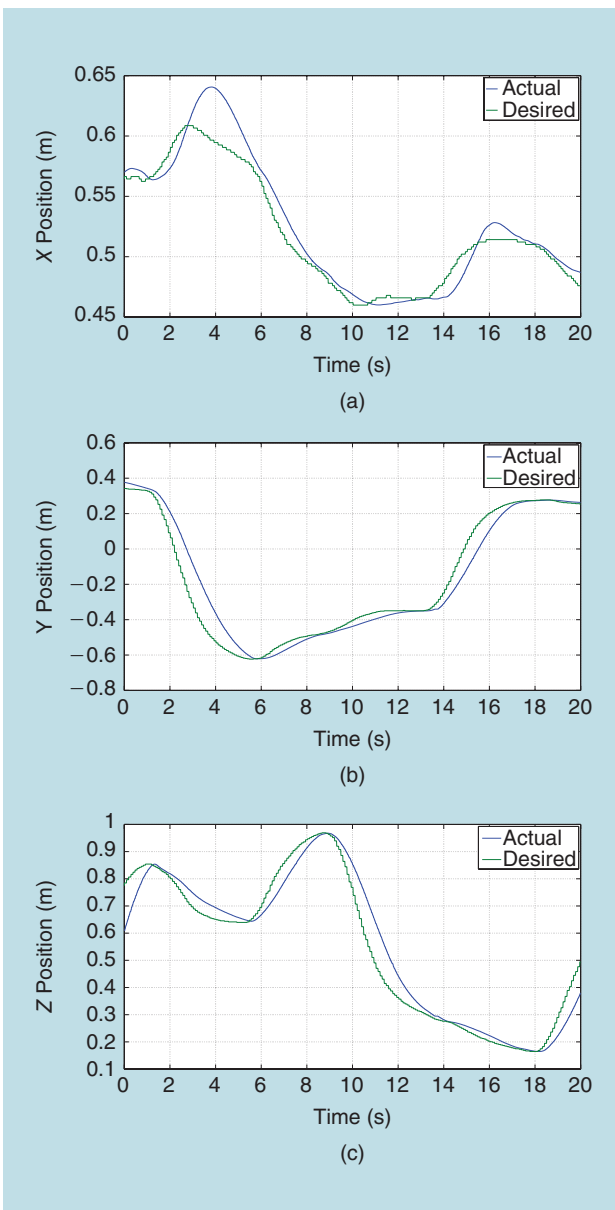


Figure 11. Case study 3: path tracking for (a) the X coordinate, (b) the Y coordinate, and (c) the Z coordinate.

developing alternative control systems. The use of smartphones and tablets in research and development is also found in other areas, as they represent a significant business opportunity for manufacturers, who need to consistently develop better hardware and operating systems. For these reasons, these applications are very interesting and appealing in the forthcoming industrial applications.

Accuracy

Accuracy refers to the possibility of positioning the robot's end-effector at a desired target point within the workspace. Concerning the second case study, the line-following experiment is performed on a randomly generated line drawn on a table. Figure 10 shows the detected line and the actual path followed by the robot's end-effector, respectively. Once the line is detected, the robot executes the movement offline in about 10 s with a maximum position error less than 5 mm. This position error could be reduced even more by increasing the sampling resolution of the detected traversable path.

Performances

Within the particular case study of the *Leap Motion* controller (case study 3), a real-time path tracking analysis of the Cartesian paths for X, Y, and Z coordinates is performed, measuring the difference between the desired and actual position of the robot's end-effector. The results are shown in Figure 11.

Moreover, to assess the communication delay of JOpenShowVar, a time-delay analysis is carried out. The considered time-delay represents the time for each message to be received, performed, and notified to the client by the KRC. In particular, this time-delay is obtained by considering the exact instant in which the request is dispatched from the client and the exact instant in which the information is received back from the client. It is not possible to exactly determine the time-delay mainly because Kuka has not released any information about it. During our experiments, a deterioration of the time-delay has usually been noticed when making the selection of a program and when the robot is engaged in some movements or there are several active interrupts. When considering the causes of the delay, it is possible to distinguish two main components that affect the access time for a variable to be either read or written from the client side as follows.

- The time interval that is required for the TCP/IP protocol to transfer the information from the client to the server and then back to the client. This time component is nondeterministic.
- The time interval that is required for the Kuka controller to acquire the information from the robot. Also, this time component is nondeterministic.

As already mentioned in the "JOpenShowVar Architecture" section, the time-delay is not affected by the kind of access to be performed (whether it is a reading or a writing operation) or by the length of the message. Therefore, it is beneficial to aggregate several variables in logical structures when reading or writing

data. Using data structures it is possible to simultaneously access several variables, thereby minimizing the access time.

Considering the third case study, a time-delay analysis is carried out for the same Cartesian paths, as shown in Figure 12. Even though there are a few spikes with a larger time interval, an average access time of 4.27 ms is obtained in this case. It should be noted that all of the considered case studies are equally affected by similar communication delays except for the second case study which is performed offline and, therefore, does not present any run-time delays.

To further assess the performances of the proposed interface with regard to the communication delay, an additional experiment is performed. In particular, the possibility of developing alternative control methods is considered (as presented in case study 3). Any custom control algorithm that does not use the standard KRC kinematics must calculate the corresponding sampling point configurations for the desired end-effector positions. In other words, each control algorithm works as a motion planner. To ensure smooth movements for the manipulators, it is necessary to generate trajectories out of these given sampling points. A well-suited trajectory is the basic prerequisite for the design of a high-performance tracking controller and ensures that no kinematic nor dynamic limits are exceeded. Such a controller guarantees that the controlled robot will follow its specified path without drifting away. Therefore, feedback control has to be applied to be able to compensate external disturbances as well as disturbances from communication time delays. Note that time data are a free parameter because, as already mentioned, the sampling time of the mapping algorithm is not constant. A possible solution for generating well-suited trajectories consists of using a proportional integral derivative (PID) controller for each joint. Different methods can be used to tune the PID parameters, such as the one proposed in [14]. The response of the PID controller that is adopted in the third case study is shown in Figure 13 for all joints of the robot. The interface provided by JOpenShowVar demonstrates a relatively fast reaction to the inputs and reasonable output error for research purposes, considering the dimension of the controlled model.

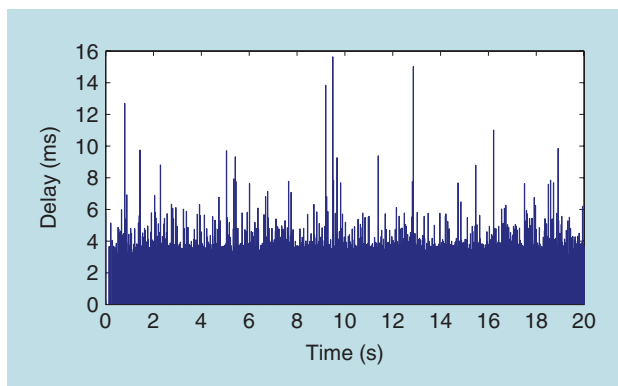


Figure 12. Case study 3: time-delay analysis for the corresponding Cartesian paths shown in Figure 11.

Effectiveness

Concerning the fourth case study, the aim is to show the possibility of operating the robot and transferring the corresponding force feedback to the operator. The plots in Figure 14(a) and (b) show the actual position for the x -, y -, and z -axes as a result of the haptic input device's movements, operated by the user, and the corresponding joint angles, respectively. In this particular case, the operator maneuvers the robot to lift the end-effector up at first, then down again with a displacement also in the x - and y -axes. In this case study, the input signal is not scaled to the robot's workspace, since the haptic device is only used to set the direction of movement for the robot and to transfer the corresponding force feedback to the operator. Even though there is a delay between the input signal and the actual position, the results show that the system is quite responsive to the user's inputs. Figure 14(c) and (d) shows the torques applied to the robot's joints and the corresponding forces applied to the robot's end-effector, respectively. The operator also perceives a force feedback that is proportional to forces applied to the robot's end-effector.

Conclusions and Future Work

This article highlights the features of JOpenShowVar as a cross-platform communication interface to Kuka industrial

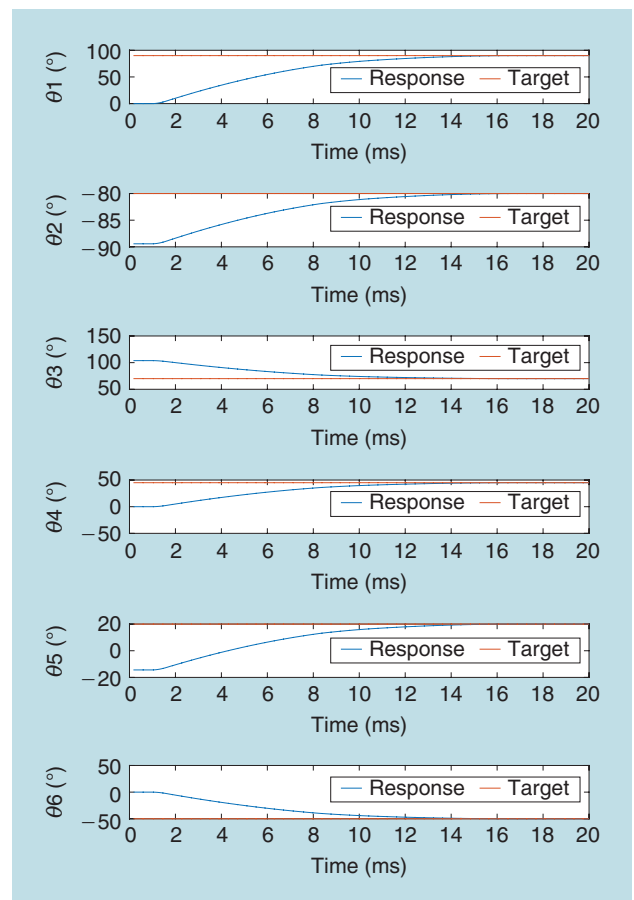


Figure 13. Target and response of the adopted PID controller for all the joints of the robot.

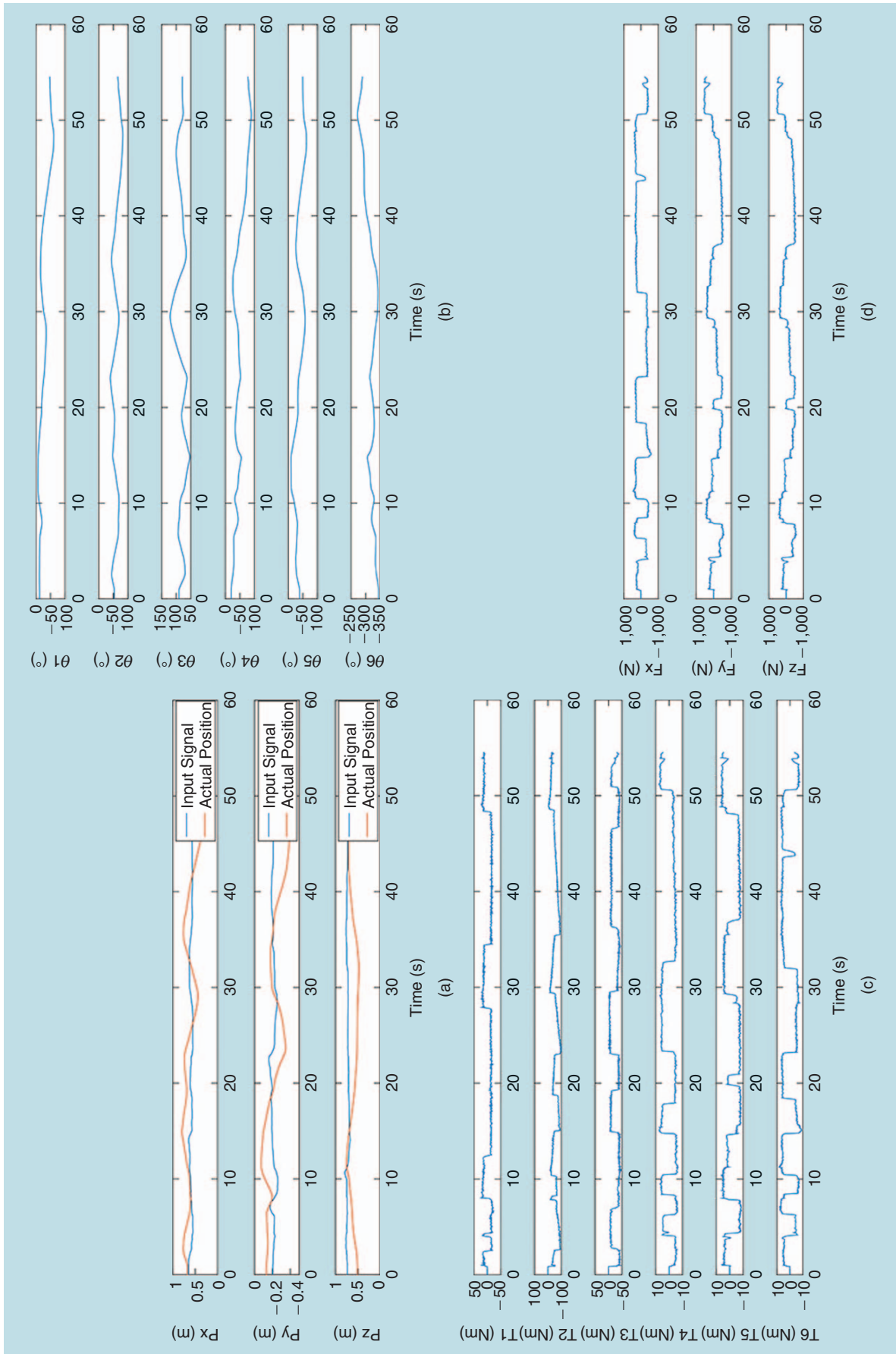


Figure 14. Case study 4: (a) the actual position for the x-, y-, and z-axes as a result of the haptic input device's movements (in this case, the input signal is not scaled to the robot's workspace, since the haptic device is only used to set the direction of movement for the robot and to transfer the force feedback to the operator), (b) the corresponding joint angles, (c) the corresponding joint torques, and (d) the corresponding forces applied to the robot's end-effector.

robots. Even though JOpenShowVar only provides a soft real-time access to the manipulator to be controlled, this middleware package opens up to a variety of possible applications making it feasible to use different input devices, sensors, and to develop alternative control methods. Special care has been devoted to keep JOpenShowVar methods intuitive and easy to use. The versatility and effectiveness of the interface have been demonstrated through four case studies. The first two case studies are open-loop applications, while the last two case studies describe the possibility of implementing closed-loop applications. Recently, our research group employed JOpenShowVar to realize a framework that makes it possible to reproduce the challenging operational scenario of controlling offshore cranes via a safe laboratory setup [15].

In the future, different control algorithms, such as the ones implemented in [16]–[18] may be tested as alternatives to the standard KRC. Finally, some effort should be put in the standardization process of JOpenShowVar to make it even more reliable for both the industrial and the academic practice. It is the opinion of the authors that the key to maximize the long-term, macroeconomic benefits for the robotics industry and for academic robotics research relies on the closely integrated development of open content, open standards and open source. In this perspective, the integration of the proposed interface with ROS is of crucial importance as a necessary future work. The community of developers at ROS is looking forward to the integration of JOpenShowVar.

Acknowledgments

This work is partly supported by the Research Council of Norway through the Centers of Excellence funding scheme, project number 223254 and the Innovation Programme for Maritime Activities and Offshore Operations, project number 217768.

References

- [1] M. Schopfer, F. Schmidt, M. Pardowitz, and H. Ritter, "Open source real-time control software for the kuka light weight robot," in *Proc. 8th IEEE World Congr. Intelligent Control Automation*, Jinan, China, 2010, pp. 444–449.
- [2] G. Schreiber, A. Stemmer, and R. Bischoff, "The fast research interface for the KUKA lightweight robot," in *Proc. IEEE ICRA Workshop Innovative Robot Control Architectures Demanding (Research) Applications Modify Enhance Commercial Controllers*, 2010, pp. 15–21.
- [3] KUKA Robotics Corp. (2014, Mar.). KUKA. [Online]. Available: <http://www.kuka-robotics.com/>
- [4] *Expert Programming*, KUKA Robotics Corp., KUKA, 2003.
- [5] H. Mühe, A. Angerer, A. Hoffmann, and W. Reif, "On reverse-engineering the kuka robot language," in *Proc. 1st IEEE/RSJ Int. Workshop Domain-Specific Languages Models Robotic Systems, Int. Conf. Intelligent Robots Systems*, 2010, pp. 16, 217, 223, and 224.
- [6] KUKA Robotics Corp. (2014, Mar.). KUKA, Hub technologies. [Online]. Available: http://www.kuka-robotics.com/en/products/software/hub_technologies/print/start.htm
- [7] F. Sanfilippo, L. I. Hatledal, H. Zhang, M. Fago, and K. Y. Pettersen, "JOpenShowVar: An open-source cross-platform communication interface to

- kuka robots," in *Proc. IEEE Int. Conf. Information Automation*, Hailar, China, 2014, pp. 1154–1159.
- [8] Google Inc. (2014, Mar.). Android. [Online]. Available: <http://www.android.com/>
- [9] Leap Motion Inc. (2014, Mar.). The leap motion controller. [Online]. Available: <http://www.leapmotion.com/>
- [10] F. Chinello, S. Scheggi, F. Morbidi, and D. Prattichizzo, "KUKA control toolbox," *IEEE Robot. Automat. Mag.*, vol. 18, no. 4, pp. 69–79, 2011.
- [11] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: An open-source robot operating system," in *Proc. ICRA Workshop Open Source Software*, vol. 3, no. 3.2, 2009, p. 5.
- [12] D. Delling, P. Sanders, D. Schultes, and D. Wagner, "Engineering route planning algorithms," in *Algorithmics of Large and Complex Networks*. Berlin, Germany: Springer-Verlag, 2009, pp. 117–139.
- [13] B. Siciliano and O. Khatib, *Springer Handbook of Robotics*. Berlin, Germany: Springer-Verlag, 2008.
- [14] I. Pan, S. Das, and A. Gupta, "Tuning of an optimal fuzzy PID controller with stochastic algorithms for networked control systems with random time delay," *ISA Trans.*, vol. 50, no. 1, pp. 28–36, 2011.
- [15] F. Sanfilippo, L. I. Hatledal, H. Zhang, W. Rekdalsbakken, and K. Y. Pettersen, "A wave simulator and active heave compensation framework for demanding offshore crane operations," in *Proc. IEEE Canadian Conf. Electrical Computer Engineering*, Halifax, Nova Scotia, Canada, 2015, pp. 1588–1593.
- [16] F. Sanfilippo, L. I. Hatledal, H. G. Schaathun, K. Y. Pettersen, and H. Zhang, "A universal control architecture for maritime cranes and robots using genetic algorithms as a possible mapping approach," in *Proc. IEEE International Conf. Robotics Biomimetics*, Shenzhen, China, 2013, pp. 322–327.
- [17] F. Sanfilippo, L. I. Hatledal, H. Zhang, and K. Y. Pettersen, "A mapping approach for controlling different maritime cranes and robots using ANN," in *Proc. IEEE Int. Conf. Mechatronics Automation*, Tianjin, China, 2014, pp. 594–599.
- [18] L. I. Hatledal, F. Sanfilippo, and H. Zhang, "JIOP: A java intelligent optimisation and machine learning framework," in *Proc. 28th European Conf. Modelling Simulation*, Brescia, Italy, 2014, pp. 101–107.
- [19] Force dimension. (2015, Nov.). Omega.7. [Online]. Available: <http://www.forcedimension.com/products/omega-7/overview>

Filippo Sanfilippo, Department of Maritime Technology and Operations, Aalesund University College, Aalesund, Norway. E-mail: fisa@hials.no.

Lars Ivar Hatledal, Department of Maritime Technology and Operations, Aalesund University College, Aalesund, Norway. E-mail: laht@hials.no.

Houxiang Zhang, Department of Maritime Technology and Operations, Aalesund University College, Aalesund, Norway. E-mail: hozh@hials.no.

Massimiliano Fago, Industrial Marking and Tracking Systems (IMTS) Società a Responsabilità Limitata (S.r.l.) Company, Taranto, Italy. E-mail: massimiliano.fago@gmail.com.

Kristin Ytterstad Pettersen, Department of Engineering Cybernetics, Norwegian University of Science and Technology, Trondheim, Norway. E-mail: kristin.y.pettersen@itk.ntnu.no.

